04-Express-API/03-post-requests.md

📩 POST Requests & Validation

🎯 Creating Resources

POST - Create New Resources

Learn to handle data creation and validate input

📝 POST Request Basics

Creating a Course

```
app.post('/api/courses', (req, res) => {
    const course = {
        id: courses.length + 1,
        name: req.body.name
    };

    courses.push(course);
    res.send(course);
});
```

Reading Request Body

To access `req.body`, we need middleware:

```
// Add at the top of your file
app.use(express.json());
```

This parses incoming JSON in request bodies.

🔌 Enable JSON Parsing

Complete Setup

```
const express = require('express');
const app = express();

// Enable JSON body parsing
app.use(express.json());

const courses = [
    { id: 1, name: 'course1' },
    { id: 2, name: 'course2' },
    { id: 3, name: 'course3' }
];

app.post('/api/courses', (req, res) => {
    const course = {
        id: courses.length + 1,
        name: req.body.name
    };

    courses.push(course);
    res.send(course);  // Return created resource
});

app.listen(3000, () => {
    console.log('Listening on port 3000...');
});
```

📮 Testing with Postman

Using Postman

1. Create Request

- Method: **POST**
- URL: `http://localhost:3000/api/courses`

2. Set Body

- Body → **raw** → **JSON**

```
{
    "name": "new course"
}
```

3. Send

Response:

```
{
    "id": 4,
    "name": "new course"
}
```

🔌 REST Client (VS Code Extension)

Alternative to Postman

Create `test.http` file:

```
### Get all courses
GET http://localhost:3000/api/courses

### Create a course
POST http://localhost:3000/api/courses
Content-Type: application/json

{
    "name": "new course"
}

### Get specific course
```

```
GET http://localhost:3000/api/courses/1
```

Click “Send Request” above each request!

⚠️ Input Validation Problem

Never Trust User Input!

What if:

- Client forgets to send `name`?
- Name is empty string?
- Name is too short?
- Name contains invalid characters?

Without Validation

```
const course = {
    id: courses.length + 1,
    name: req.body.name  // Could be undefined, empty, or invalid!
};
```

⚠️ This will cause problems!

✅ Basic Validation

Manual Validation

```
app.post('/api/courses', (req, res) => {
    // Validate
    if (!req.body.name || req.body.name.length < 3) {
        // 400 Bad Request
        return res.status(400).send('Name required, min 3 chars');
    }
```

```
    const course = {
        id: courses.length + 1,
        name: req.body.name
    };

    courses.push(course);
    res.send(course);
});
```

Problems with Manual Validation

- ❌ Gets complex quickly
- ❌ Repetitive code
- ❌ Hard to maintain
- ❌ Doesn't scale well

🎯 Validation with Joi

Install Joi

```
npm i joi
```

Import and Use

```
const Joi = require('joi');

app.post('/api/courses', (req, res) => {
    // Define schema
    const schema = Joi.object({
        name: Joi.string().min(3).required()
    });

    // Validate
    const result = schema.validate(req.body);
    console.log(result);

    // ... rest of code
```

```
});
```

🔍 Joi Validation Result

With Invalid Input

Send: `{ "name": "n" }`

Console output:

```
{
  value: { name: 'n' },
  error: [Error [ValidationError]: "name" length must be at least 3 characters long] {
    _original: { name: 'n' },
    details: [ [Object] ]
  }
}
```

With Missing Name

Send: `{ "name": "" }`

Console output:

```
{
  value: { name: '' },
  error: [Error [ValidationError]: "name" is not allowed to be empty]
}
```

✅ Complete Joi Validation

Full Implementation

```
app.post('/api/courses', (req, res) => {
    // Define validation schema
    const schema = Joi.object({
        name: Joi.string().min(3).required()
    });

    // Validate request body
    const result = schema.validate(req.body);

    // Check for validation errors
    if (result.error) {
        return res.status(400).send(result.error.details[0].message);
    }

    // Create course
    const course = {
        id: courses.length + 1,
        name: req.body.name
    };

    courses.push(course);
    res.send(course);
});
```

📋 User-Friendly Error Messages

Extracting the Message

Instead of sending the entire error object:

```
// ❌ Too much information
res.status(400).send(result.error);
```

Send just the message:

```
// ✅ Clean error message
```

```
res.status(400).send(result.error.details[0].message);
```

Examples

Empty name:

```
"name" is not allowed to be empty
```

Too short:

```
"name" length must be at least 3 characters long
```

💡 Joi Schema Examples

Different Validation Rules

```
// String with min/max length
const schema = Joi.object({
    name: Joi.string().min(3).max(50).required()
});

// Number with range
const schema = Joi.object({
    age: Joi.number().integer().min(0).max(120).required()
});

// Email
const schema = Joi.object({
    email: Joi.string().email().required()
});

// Multiple fields
const schema = Joi.object({
    name: Joi.string().min(3).required(),
    email: Joi.string().email().required(),
    age: Joi.number().integer().min(18)
});

// Optional field
const schema = Joi.object({
    name: Joi.string().min(3).required(),
    description: Joi.string().optional()
```

```
});
```

🎯 Best Practices

POST Request Guidelines

✅ **DO:** Always validate input
✅ **DO:** Return the created resource
✅ **DO:** Use 400 status for validation errors
✅ **DO:** Send user-friendly error messages
✅ **DO:** Use Joi for complex validation

❌ **DON'T:** Trust user input
❌ **DON'T:** Expose sensitive error details
❌ **DON'T:** Forget `app.use(express.json())`
❌ **DON'T:** Create resources without validation

📊 HTTP Status Codes for POST

Status	Meaning	When to Use
200 OK	Success	Resource created successfully
201 Created	Resource created	Preferred for POST (best practice)
400 Bad Request	Invalid input	Validation failed
401 Unauthorized	Not authenticated	Login required
403 Forbidden	Not allowed	No permission

🏠 Course Home | 📘 Chapter 4 Home

← Previous: GET Requests | Next: PUT & DELETE →